

OpenCV Image Processing Lab

Contour-Based Circle Vision · No Hough Transform

Project Documentation | Diploma in Artificial Intelligence and Machine Learning | GTTC Belagavi

1. Introduction

The OpenCV Image Processing Lab is a web-based image processing and computer vision toolkit that lets a user perform classical OpenCV operations from a browser instead of writing code for every operation. An image is uploaded once, and every supported operation can then be applied and compared from a single interactive dashboard. The project builds on an earlier contour-based circle detection tool — that original detector, its API, and its algorithm are preserved unchanged, while a full Image Processing Lab and a Shape Detection module have been added on top of it.

2. Problem Statement

Learning image processing is easier when the effect of an operation can be seen immediately, but most OpenCV learning is done by editing and re-running scripts for every small change. There was a need for a single interactive interface where an image can be processed with different operations, filters, and parameters, and the original and processed images can be compared side by side without touching code.

3. Objective

- Provide an interactive web interface for common OpenCV operations, with original and processed images shown side by side.
- Display the exact processing stages and OpenCV calls used for each operation, for learning purposes.
- Detect geometric shapes and circles using contour geometry, without `cv2.HoughCircles()`.
- Allow the processed result to be downloaded or converted to another image format.

4. Technologies Used

Layer	Technology
Frontend	React + TypeScript + Vite
Backend	Python + FastAPI (Uvicorn server)
Computer Vision / Numerical	OpenCV (cv2), NumPy
API Data Validation	Pydantic

5. Key Concepts Covered

The project covers image loading and preview, grayscale conversion, binary thresholding, image filtering (Average, Gaussian, Laplacian sharpening), edge detection (Sobel, Canny), morphological operations (Dilation, Erosion, Opening, Closing), contour-based shape and circle detection, and side-by-side result visualization with download.

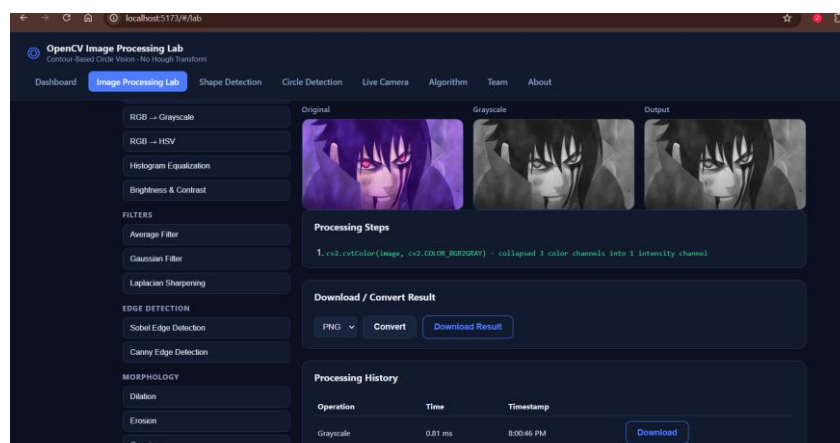


Image Processing Basics

Before filtering and edge detection can be applied, an image usually needs to be prepared. This page covers the basic conversions the project performs before deeper processing.

1. Grayscale

Grayscale conversion turns a colour image into shades of gray by collapsing the three colour channels into a single brightness channel. Most other operations, such as thresholding and edge detection, work on a single channel, so grayscale is usually the first processing step.

2. Binary Image

A binary image is created by applying a threshold value to a grayscale image: pixels above the threshold become white (foreground), and pixels below it become black (background). This separates the object of interest from the rest of the image.

3. RGB → Grayscale

OpenCV stores colour images in BGR order rather than RGB. This operation explicitly corrects the channel order before converting the image to grayscale, so colours are interpreted correctly.

4. RGB → HSV

This converts the image from colour (Red-Green-Blue) representation to HSV — Hue (colour), Saturation (colour intensity), and Value (brightness) — which is often easier to use for colour-based analysis.

5. Brightness and Contrast

Brightness is adjusted by shifting every pixel value up or down, while contrast is adjusted by scaling pixel values. Together, they let the user control how light, dark, or vivid the image appears.

Basic Processing Workflow

Input Image → Grayscale → Binary / Enhanced Image → Processing → Output Image

For any multi-step operation, the interface shows every intermediate stage rather than just the final result, as in the Sobel edge-detection pipeline below.

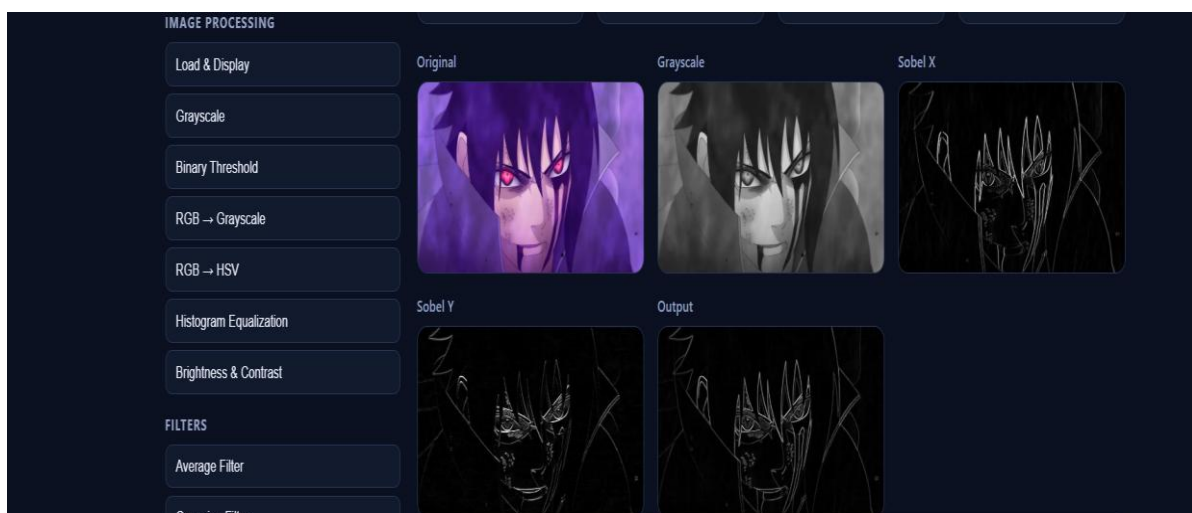


Figure 2: Algorithm Steps

Image Filtering

Filtering is used to smooth an image, most often to reduce noise before further operations such as edge detection. The project provides two main smoothing filters.

1. Average Filter

The average filter smooths an image by replacing each pixel with the average value of its neighbouring pixels. It is mainly used for basic noise reduction and smoothing. A simple 3×3 averaging kernel, where every neighbour contributes equally, looks like this:

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

Each pixel in the output is the sum of the 9 neighbouring pixels multiplied by 1/9, i.e. their average.

2. Gaussian Filter

The Gaussian filter also smooths the image, but instead of weighting every neighbouring pixel equally, it gives higher weight to pixels closer to the centre using a Gaussian curve. This produces smoother, more natural-looking results than a plain average filter, and is commonly applied before edge detection to reduce false edges caused by noise. The project applies this using:

```
cv2.GaussianBlur(image, (19, 19), 0)
```

Filtering Workflow

Original Image → Average / Gaussian Filter → Noise Reduction → Smoother Image → Further Processing

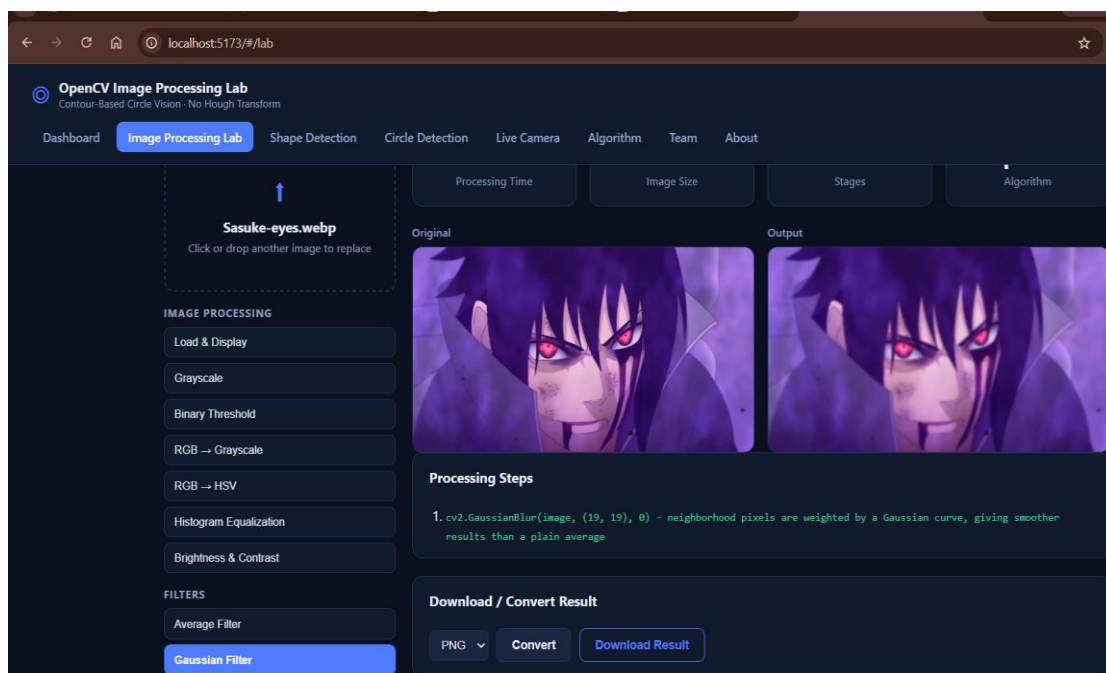


Figure 3: Gaussian Filtering Result

Edge, Morphology and Shape Detection

1. Sobel Edge Detection

Sobel edge detection finds changes in pixel intensity, which usually correspond to the boundaries of objects. Sobel X detects vertical edges (horizontal intensity change) and Sobel Y detects horizontal edges (vertical intensity change). Together, they highlight the important boundaries in an image.

2. Canny Edge Detection

Canny edge detection is a multi-stage edge detector that produces a clean, well-connected binary edge map using two threshold values (a low and a high threshold). The project applies it as:

```
cv2.Canny(blurred, 80, 240)
```

3. Morphological Dilation and Erosion

Dilation and erosion work on binary images and adjust the size of the white (foreground) regions.

Operation	Effect
Dilation	Expands / grows foreground regions, helping to connect or strengthen them
Erosion	Shrinks foreground regions, removing small noise specks

Combining the two gives two further operations already available in the project: Opening (erosion then dilation, removes noise without shrinking real shapes) and Closing (dilation then erosion, fills small holes without growing the shape).

4. Contour and Circle / Shape Detection

A contour is the boundary line of an object in an image. Once contours are found, each one can be analysed using geometric properties such as area, perimeter, number of vertices, and circularity to decide what shape it represents.

Instead of using `cv2.HoughCircles()`, this project detects circles by fitting a minimum-area rectangle to each contour, calculating its centre, and measuring how far every contour point lies from that centre. If most points lie at nearly the same distance from the centre, the contour is classified as a circle — a purely geometric, contour-based approach.

Shape / Circle Detection Flow

Image → Grayscale → Filtering → Edge Detection → Contours → Shape Analysis → Detected Shape

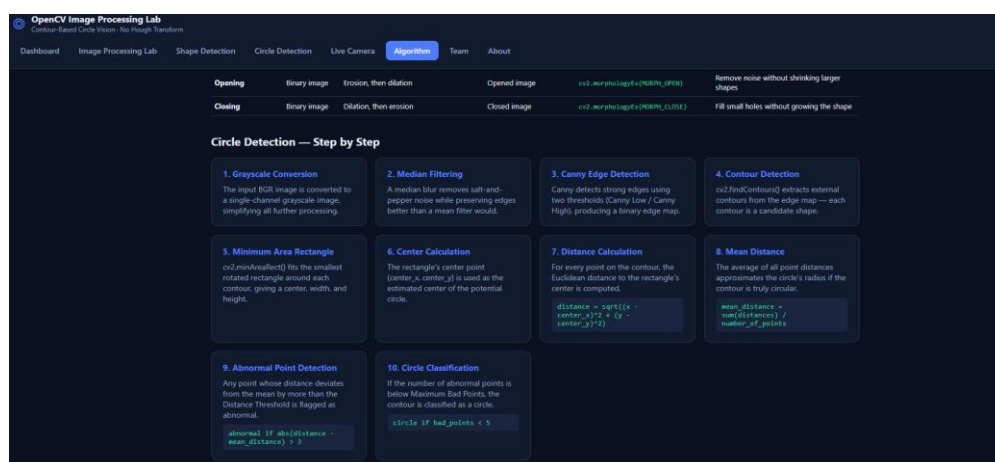


Figure 4: Circle Detection Processing

Project Working, Results and Conclusion

Working of the Web Application

- Load Image — the user uploads an image from their device.
- Select Processing Operation — an operation is chosen from the sidebar (Image Processing, Filters, Edge Detection, or Morphology).
- Process Image — the backend applies the selected OpenCV operation and returns the result.
- Display Original and Output — both images are shown side by side for comparison.
- View Processing Steps — the exact OpenCV calls and processing stages used are listed.
- Save / Download Result — the processed image can be converted and downloaded as PNG, JPG, or BMP.

Start → Load Image → Select Operation → Apply Processing → Display Result → Save / Download → End

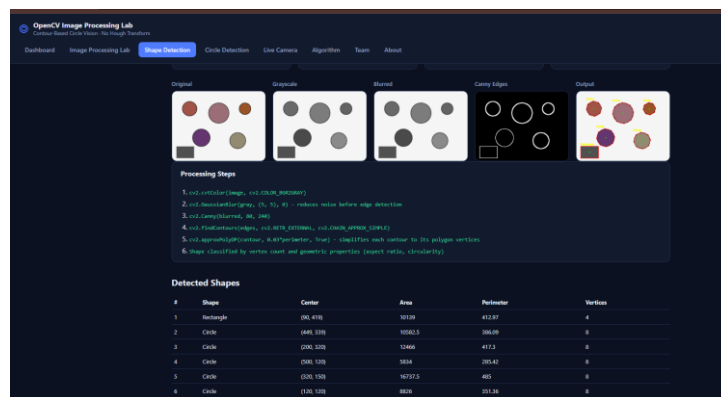


Figure 5: Shape Detection Result

Results

- The project successfully demonstrates basic image-processing operations through a live web interface.
- Original and processed images are displayed together for direct visual comparison at every stage.
- Shape and circle detection results are displayed along with geometric properties such as area, perimeter, and vertex count.

Advantages

- Simple, browser-based interface — no coding needed to try an operation.
- Multiple image-processing operations, all visually compared side by side from one dashboard.
- Useful for learning OpenCV and image-processing concepts.

Limitations

- Mainly intended for basic, classical image-processing operations.
- Advanced AI-based image understanding is not implemented, and detection accuracy depends on image quality and the chosen parameters.

Future Scope

- More advanced filters, real-time camera processing, and additional shape-detection techniques with a formal circularity metric.
- AI/ML-based object detection and image analysis.